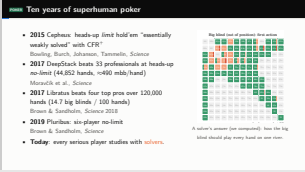


Timing plan (60 min, 40 slides). Hook & rules 6 · Games, equilibria, sequence form 11 · Regret minimisation and CFR 13 · What solutions look like 6 · The new method 13 · Demo, results, limits, wrap-up 11.
If running late, skip: "Kuhn poker", "Modern CFR = numerical tricks" (one sentence), "Attempt 1: Anderson" (one sentence), "The linear algebra" (one sentence: quasi-definite, so any ordering works), backup slides.

Opening line: "Ten years ago the best poker players in the world lost to programs whose core is a few lines of arithmetic repeated a few billion times. This talk is about that arithmetic, read as numerical analysis — and about what a numerical analyst can add to it."

The slides alternate between POKER (green tag) and MATH (blue tag); orange NEW marks our own contribution. Everything labelled "we computed" was produced by the Python solver that accompanies the talk (repository `poker-solver`); everything else is quoted from the cited papers.

POKER Ten years of superhuman poker

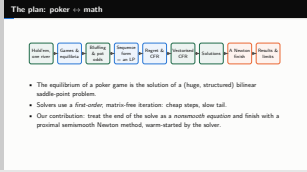


The common core of all these programs is an iterative method — counterfactual regret minimisation — applied to a gigantic saddle-point problem. Libratus and DeepStack add *abstraction* (shrinking the game) and *subgame solving* (re-solving the part of the game you are actually in), but the inner loop is still a CFR-type method.

"Solvers" (PioSolver, GTO+, GTO Wizard, ...) are what human players use: you describe one spot — the board, the two ranges, the allowed bet sizes — and the program computes an equilibrium of that subgame. The picture on the right is exactly such an output, produced by our own solver for a river spot we will meet again.

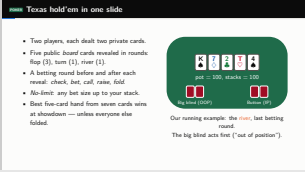
mbb/hand = thousandths of a big blind per hand.

The plan: poker ↔ math



Say what the audience will get: (1) enough poker to read a solver output, (2) the mathematics of why CFR converges and why it is fast in practice, (3) a new method — from the numerical-optimisation toolbox, not the game-theory one — with honest results: where it wins by orders of magnitude, and where it does not (yet).

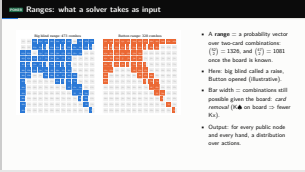
POKER Texas hold'em in one slide



Keep this light — most of the audience has played or watched poker. The key structural fact: the board is *public*, the two hole cards are *private*. Strategy must therefore be a function of (public history, your own cards), and you do not know the opponent's cards — only a *probability distribution* over them, which poker players call a **range**.

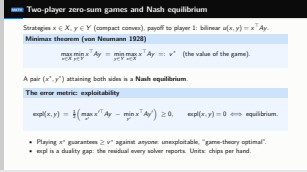
The river spot is fixed for the whole talk: board K♠ 7♢ 2♣ T♡ 4♠, pot 100, both players have 100 behind. Big blind may check or bet 33%, 75% of pot or all-in; facing a bet you may fold, call or go all-in.

POKER Ranges: what a solver takes as input



The 13-by-13 grid is the standard poker picture: pairs on the diagonal, suited hands above, offsuit below. Each cell is a class (e.g. "AKs" $= 4$ combinations). Mathematically the input is just two nonnegative vectors $r_1 \in \mathbb{R}^{n_1}$, $r_2 \in \mathbb{R}^{n_2}$, and card removal is a $0/1$ compatibility matrix C between them — both will appear in every formula from now on.

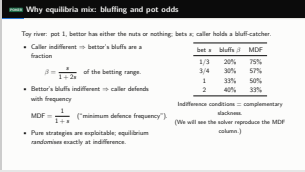
MATH Two-player zero-sum games and Nash equilibrium



For an applied-math audience: this is a convex–concave saddle-point problem with a bilinear coupling; exploitability is the primal–dual gap. Everything later is a method for driving this gap to zero.

Practical solvers stop at an exploitability of a fraction of a percent of the pot; we will push to 10^{-13}. Our numbers are in chips with pot $= 100$, so 10^{-2} chips is 10^{-4} of the pot.

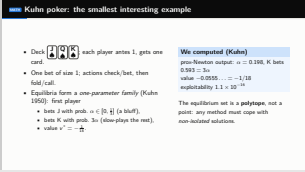
POKER Why equilibria mix: bluffing and pot odds



Derivation (say it, don't write it). Caller with a bluff-catcher: calling wins $1+s$ against a bluff and loses s against value; indifferent when $\beta(1+s) = (1-\beta)s$, i.e. $\beta = s/(1+2s)$. Bettor with air: betting wins the pot 1 when the caller folds and loses s when called; checking wins nothing; indifferent when $(1-c)\cdot 1 = c\,s$, i.e. $c = 1/(1+s)$.

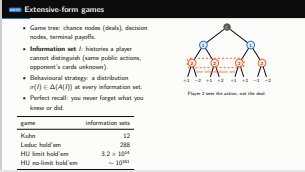
The mathematical point: at an equilibrium, every action played with positive probability is a best response — this is complementary slackness in the LP we are about to write down. Mixing happens exactly where reduced costs vanish.

MATH Kuhn poker: the smallest interesting example



Kuhn poker is the poker analogue of a 2×2 test matrix. The one-parameter family matters for numerical analysis: the solution set of the equilibrium problem is a polyhedron, generally not a point, so Jacobians at a solution are singular and textbook Newton theory does not apply as-is. Our method handles this by regularisation (proximal point) — which also picks one particular member of the family (here $\alpha \approx 0.198$).

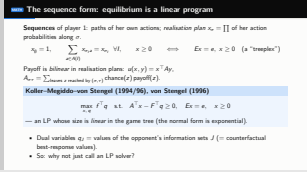
MATH Extensive-form games



Leduc hold'em (Southey et al. 2005) is the standard research benchmark: a six-card deck (J, Q, K in two suits), one private card each, one public card, two betting rounds with bets 2 and 4 and at most two raises per round. 288 information sets.

The size column is the reason the subject exists: one cannot write down HUNL. Real systems solve an *abstraction* (similar hands and bet sizes merged) and then re-solve subgames like our river spot during play.

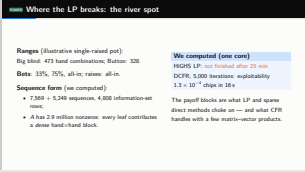
MATH The sequence form: equilibrium is a linear program



This is the key modelling step. Mixed strategies over pure strategies of the normal form would have exponentially many variables; realisation plans have one variable per sequence, and perfect recall makes the payoff bilinear in them.

E and F are very sparse (one row per information set, entries ± 1) and encode flow conservation on a tree. A is where the poker lives. Remember the dual variables: they are *values* of information sets, i.e. the same quantities CFR computes every iteration — this is how we will warm-start the duals later.

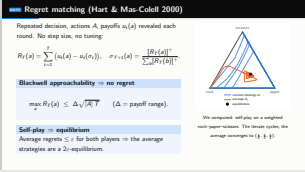
POKER Where the LP breaks: the river spot



This is the honest motivation for first-order methods: for subgame-sized problems with full ranges, the constraint matrix is not "sparse" in any useful sense, because each terminal couples every hand of one player to every compatible hand of the other. The LP solver run (HiGHS via SciPy, default method) shared the machine with another job; we stopped it after 25 minutes.

On the smaller test spots the LP is perfectly fine (Leduc: 0.04 s, small river: 1.6 s) — we will show those numbers too.

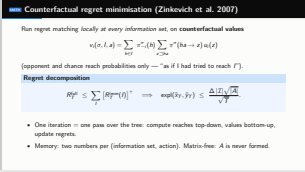
MATH Regret matching (Hart & Mas-Colell 2000)



Proof idea of the bound: the regret vector R_T satisfies $\|[R_T]^+\|^2 \le \|[R_{T-1}]^+\|^2 + \Delta^2|A|$ because the cross term $\langle [R_{T-1}]^+, r_T\rangle$ vanishes by the choice of σ_T — the instantaneous regret is orthogonal to the current positive regrets. Sum and take square roots.

The folk theorem: average regret ε for both players means $\max_{x'} x'^{\top} A\bar{y} - \min_{y'} \bar{x}^{\top} A y' \le 2\varepsilon$, because the average of the realised payoffs is sandwiched.

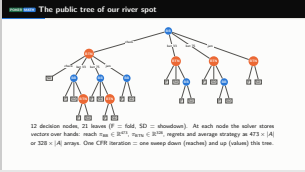
MATH Counterfactual regret minimisation (Zinkevich et al. 2007)



The decomposition is the whole trick: a global regret over an exponential strategy space is bounded by a sum of small, local regrets, each minimised by plain regret matching. This is why CFR scales: cost per iteration is linear in the tree, storage is linear in the number of information-set actions.

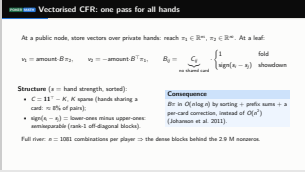
In linear-algebra language one CFR iteration computes Ay and $A^\top x$ (with x, y the current realisation plans) by tree traversals — the same two products a first-order saddle-point method needs.

POKER MATH The public tree of our river spot



This is the bridge between the game-theory picture (a tree with information sets) and the implementation: in poker the information sets at a public node are indexed by the acting player's private hand, so "all information sets at this node" is a vector, and every CFR operation becomes a vector operation. The leaves are where the opponent's range enters, through the matrices on the next slide.

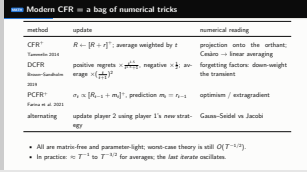
POKER MATH Vectorised CFR: one pass for all hands



This is the poker-specific numerical linear algebra. Each terminal block is "low rank plus sparse": the fold block is rank one minus a sparse card-removal correction; the showdown block, after sorting hands by strength, is a sign matrix (a semiseparable matrix — think of the inverse of a bidiagonal matrix) times the compatibility mask. Products with it are prefix sums.

For our sizes (a few hundred to 1081 hands) a dense BLAS product is already fast, so our code simply uses dense B; but this structure is what makes river solvers fast in practice, and it is the structure a scalable Newton method would have to exploit (end of talk).

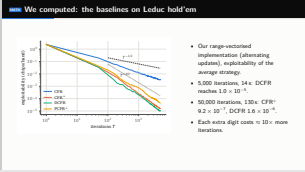
MATH Modern CFR = a bag of numerical tricks



CFR$^+$ is what solved limit hold'em. DCFR with $(\alpha, \beta, \gamma) = (1.5, 0, 2)$ is the default in many solvers. PCFR$^+$ connects regret matching to optimistic mirror descent (Farina, Kroer, Sandholm, AAAI 2021).

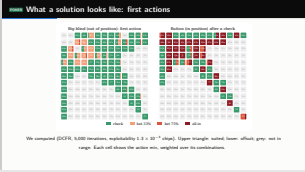
The empirical rates are much better than the $T^{-1/2}$ worst case — which is one of the open theoretical questions in the area — but they are still sublinear, and that is the gap our method addresses.

MATH We computed: the baselines on Leduc hold'em



Exploitability is in chips per hand (Leduc antes are 1 chip). The ordering CFR $\ll$ PCFR$^+$ $<$ CFR$^+$ $\approx$ DCFR is the usual one in the literature for Leduc. Note the tail: to go from 10^{-5} to 10^{-6} took ten times the work.

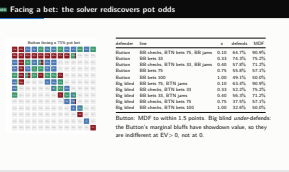
POKER What a solution looks like: first actions



Read the grid like a solver user. Big blind (left): checks most of the range, uses a small 33% bet with medium hands (Q-x, J-x type hands) — a "block bet" — and almost never bets big. Button after a check (right): *polarised* — jams the very strong hands (sets, KQ+, AK) and a set of bluffs chosen by card removal (A5s, A6s, 65s, 86s . . .), checks back the medium hands that win at showdown.

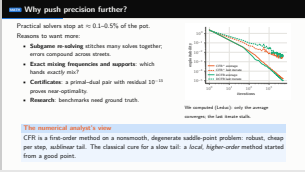
Aggregate frequencies (we computed): big blind checks 82%, bets 33% pot 16%; Button after a check jams 44%, checks 50%.

POKER Facing a bet: the solver rediscovers pot odds



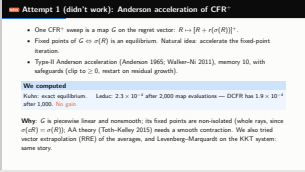
This is a nice consistency check of the whole pipeline. When the big blind bets, the Button defends 74.3% vs 75.2%, 55.8% vs 57.1%, 49.1% vs 50.0% — the toy-game formula is almost exact. When the Button bets after a check, the big blind folds much more than the MDF: we checked that the Button's weakest hands mix between checking and jamming, so the indifference that pins the defence frequency is "EV(jam) = EV(check) > 0", not "EV(jam) $= 0$". The toy model's assumption (bluffs have no showdown value) fails, and the solver tells you by how much.

MATH Why push precision further?



Make the case without overselling: for a human studying a spot, 0.3% of the pot is plenty. Precision matters for machines (re-solving, evaluation) and for science (knowing the exact equilibrium, e.g. the exact bluffing frequencies, or whether a hand mixes at all). The analogy for this audience: CFR is like a stationary iterative method — fantastic for the first digits, hopeless for the last ones — and the obvious idea is to switch to a Newton-type method near the solution.

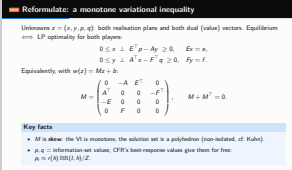
NEW Attempt 1 (didn't work): Anderson acceleration of CFR$^+$



Include this slide: audiences of numerical analysts appreciate negative results, and it motivates the reformulation. The lesson we drew: do not accelerate the regret dynamics; go back to the optimality conditions themselves and fix their two defects — non-uniqueness (handled by a proximal term) and nonsmoothness (handled by a semismooth Newton method on a smoothed-at-the-kink reformulation).

Also tried (we computed, not shown): reduced-rank extrapolation of the sequence of DCFR averages gave no improvement; plain semismooth Levenberg–Marquardt on the min-function KKT system stalled on Leduc (degenerate complementarity).

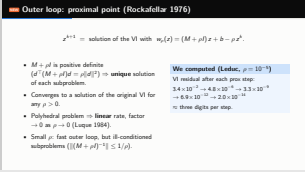
NEW Reformulate: a monotone variational inequality



This is a mixed linear complementarity problem. The reduced cost $E^\top p - Ay$ of a sequence is its counterfactual *regret*, scaled by reach: complementarity says "play only actions with zero regret". So CFR and this system talk about the same quantities — CFR iterates give us x, y (average strategies in sequence form) and, via one best-response pass, p, q.

Skewness: the x–y block is the zero-sum coupling; the (x, p) and (y, q) blocks are constraint/multiplier pairs, arranged with signs so that $M^\top = -M$.

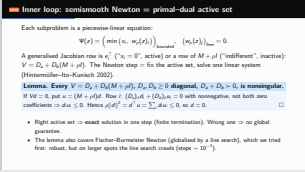
NEW Outer loop: proximal point (Rockafellar 1976)



The proximal term fixes non-uniqueness: each subproblem has exactly one solution, so Newton's method has a well-defined, nonsingular target. For polyhedral monotone problems the inverse operator is upper Lipschitz (Robinson 1981), which gives linear convergence of the proximal point method with a factor that tends to zero as $\rho \to 0$ (Luque 1984; Rockafellar 1976 for the general theory).

The trade-off is the whole story of the last part of the talk: small ρ makes the outer loop fast and the inner problems nearly as ill-posed as the original one.

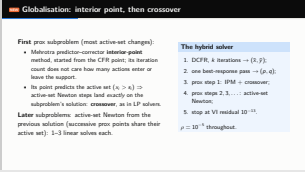
NEW Inner loop: semismooth Newton = primal–dual active set



The min-function reformulation turns complementarity into a piecewise-linear equation; semismooth Newton on it is exactly the primal–dual active-set method: guess which actions are dropped, impose indifference on the rest, solve. The lemma is the reason for the proximal term: without ρ, $d^\top M d = 0$ and the argument fails — Newton matrices of the unregularised problem are singular at degenerate solutions, which is what we saw with Levenberg–Marquardt.

Fischer–Burmeister ($\varphi(a, b) = a + b - \sqrt{a^2 + b^2}$) gives a C^1 merit function and a globally convergent damped Newton method (De Luca–Facchinei–Kanzow 1996). It worked on Leduc and the small river (about 50–70 linear solves) but on the medium spot it took steps of size 10^{-3} for dozens of iterations. That is what pushed us to the interior-point start on the next slide.

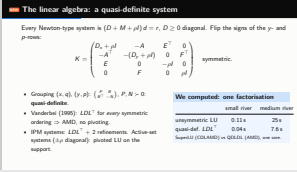
NEW Globalisation: interior point, then crossover



The division of labour mirrors what commercial LP solvers do (interior point, then crossover to a vertex), but inside a proximal loop and with a game-theoretic warm start. In our runs the interior-point phase took 6–8 factorisations on Leduc and the small river, the crossover a few active-set steps, and every later proximal step 1–3 active-set steps.

If the crossover fails (the predicted active set is wrong and the residual grows), we keep the interior point and solve the next subproblem the same way — this fallback is what happens on the medium spot.

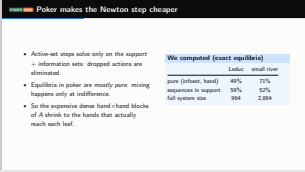
NEW The linear algebra: a quasi-definite system



This is the numerical-linear-algebra heart of the method. Row-scaled, the Newton matrix is "positive diagonal plus skew"; flipping the signs of half the rows makes it symmetric, and the sign pattern of the diagonal is exactly that of a quasi-definite matrix. Vanderbei's theorem says every symmetric permutation has an $LDL^\top$ factorisation, so a fill-reducing ordering can be chosen before seeing the numbers — the same reason interior-point codes like quasi-definite KKT systems.

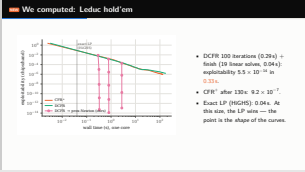
Caveat we hit: with only $\pm\rho = \pm 10^{-5}$ on the diagonal (the pure active-set systems) static pivoting is too unstable, so there we use a pivoted sparse LU, which is cheap because the active-set system lives on the support only.

POKER NEW Poker makes the Newton step cheaper



Mixing happens only at indifference, which is non-generic: in the exact Leduc equilibrium about half of the reachable (information set, hand) pairs play a single action; on the small river spot 71%. About half of all sequences are outside the support, so active-set systems are roughly half the size of the full one — and much sparser, because the payoff blocks only couple hands that reach a leaf with positive probability.

NEW We computed: Leduc hold'em

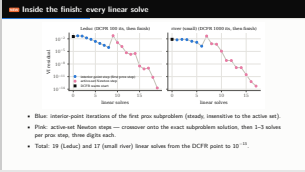


Each pink curve starts where the DCFR warm start stopped (100, 300 or 1000 iterations) and then drops vertically: three digits per proximal step. Starting from 100 DCFR iterations is enough; starting later only costs DCFR time.

The game value we obtain, $-0.085606424078\ldots$ chips for the first player, agrees with the LP to all printed digits.

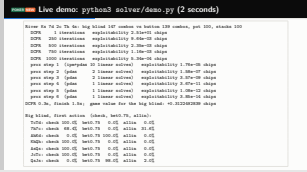
Be upfront about the LP: Leduc is tiny (964 unknowns); a simplex method finishes in 40 ms. The question is what happens when the LP does not fit — next slides.

NEW Inside the finish: every linear solve



The VI residual of the original problem is plotted after every linear solve. The interior point has a small residual in the $\min(x, w)$ sense only because it is interior; the crossover step lands on the exact solution of the first *regularised* subproblem, whose residual is $\rho\|z^1 - z^0\|$ — hence the jump. From then on each proximal step is solved exactly in one to three active-set steps.

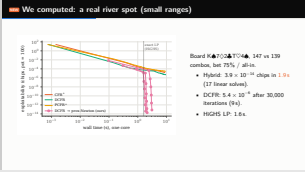
POKER NEW Live demo: `python3 solver/demo.py` (2 seconds)



Run it live if the room allows (Python 3 with numpy, scipy, qdldl). The output above is a captured run. Points to make: DCFR gains a digit per few hundred iterations; the first prox step (interior point + crossover, about ten linear solves) gains more than an order of magnitude, and each later prox step costs one or two linear solves and gains about two digits.

Poker reading: QJs (no pair, no showdown value) is the bluff — it bets; AK bets for thin value/protection; sets of tens check (to check-raise); 77 mixes check and jam — an exact mixed strategy, which the Newton finish computes to machine precision.

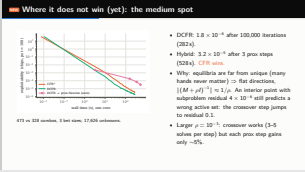
NEW We computed: a real river spot (small ranges)



Same picture on a genuine no-limit river subgame with card removal and a non-trivial betting tree. The hybrid reaches machine precision, about 10^8 times more accurately than DCFR, in a fraction of the time; DCFR's tail would need millions of iterations for the same accuracy. Here the LP is about as fast.

Exploitability in chips with pot 100: 10^{-6} chips is 10^{-8} of the pot.

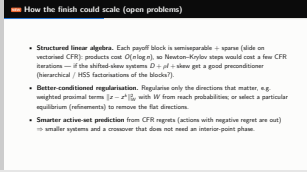
NEW Where it does not win (yet): the medium spot



State the limitation plainly. On the medium spot each factorisation of the full system costs about 8 s (quasi-definite $LDL^\top$; 25 s with plain sparse LU), i.e. thousands of DCFR iterations, and the finish does not reach its fast regime: the crossover from the interior point lands on a wrong active set (the error is amplified by $\|(M+\rho I)^{-1}\| \approx 1/\rho$), so every proximal step falls back to a full interior-point solve. We also tried decreasing ρ gradually from 10^{-3}: the active-set steps track the solution down to $\rho \approx 10^{-4}$ and then lose it.

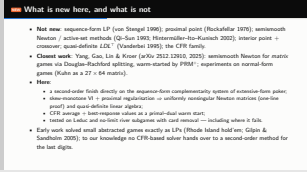
So: at realistic sizes CFR remains the right tool for the practical accuracy range, and the second-order finish is a small-to-medium-size, high-precision tool — for now.

NEW How the finish could scale (open problems)



These are honest research directions, not results. The first is the one I would bet on: the poker-specific structure that makes CFR fast (sorting, prefix sums, card-removal corrections) is exactly the structure a fast direct or preconditioned iterative solver for the Newton systems would need.

NEW What is new here, and what is not



Be precise about novelty. The general idea "first-order warm start, then semismooth Newton" for zero-sum games was proposed for matrix games by Yang, Gao, Lin and Kroer in December 2025; their Newton system lives on the normal form and they do not treat extensive-form structure. Our contribution is the extensive-form, sequence-form version with a different regularisation (proximal point on the skew VI), the quasi-definite linear algebra, a CFR-based dual warm start, and the poker experiments. Commercial and research poker solvers we know of (PioSolver, GTO+, Libratus, DeepStack) are first-order (CFR variants).

Takeaways

1. A poker equilibrium is the solution of a structured bilinear saddle-point problem; the sequence form makes it an LP of size linear in the tree.
2. CFR = regret matching at every information set: a matrix-free, nonsmooth first-order method with a provable $O(T^{-1/2})$ rate and a much better practical one — plus numerical tricks (projection, discounting, optimism, Gauss–Seidel).
3. The solver's output is readable poker: polarised betting, block bets, pot-odds defence (to within 1.5%).
4. Read the equilibrium conditions as a **skew-monotone VI**: proximal point + semismooth (active-set) Newton, started by an interior-point crossover, turns CFR's slow tail into ~20 linear solves and reaches 10^{-14} — on small and mid-sized spots.
5. At realistic sizes CFR still wins; scaling the finish is a structured linear-algebra and conditioning problem.

Code: `github.com/deanguambers/Poker-Solver` — solver, benchmarks, figures, these slides.

End with the invitation: the open problem on the previous slide is a natural one for this club — a preconditioner for shifted skew systems whose off-diagonal blocks are semiseparable-plus-sparse.

Backup: implementation summary

`games.py`	Kuhn, Leduc, no-limit river builder (7-card evaluator, card removal, range notation)
`cfr.py`	range-vectorised public-tree CFR / CFR$^+$ / DCFR / PCFR$^+$, alternating updates, best response, exploitability
`seqform.py`	sequence form (E, F, A), dual estimates from best responses; `PrecIPMPDAS` (the method of the talk), FB variants, earlier attempts
`lp.py`	reference LP via HiGHS
`aa.py`, `extrap.py`	the negative results (Anderson, RRE)
`bench_*.py`, `fig_*.py`	every number and figure in these slides

- Exploitability: $\frac{1}{2}(\mathrm{BR}_1 + \mathrm{BR}_2)$ by exact best-response passes, normalised by the deal weight Z.
- Sequence form checked against the LP: Leduc value -0.085604240780 (both).
- Dual warm start: $p_i \propto r_i(h)\,\mathrm{BR}_i(I.h)/Z$, $q_i \propto -r_i(h)\,\mathrm{BR}_i(J.h)/Z$.

Backup only.

Backup only.

Backup: effect of the proximal parameter

We computed (Fischer–Burmeister variant; Leduc, warm start DCFR 200 iterations, final exploitability $\leq 10^{-10}$ in all cases):

μ	10^{-1}	10^{-2}	10^{-3}	10^{-4}
prox steps	19	6	4	3

Continuation ($\rho_\downarrow$) did not reduce the total number of Newton steps on the small river spot (54 fixed vs 63–96).